

Stalkr

stalkr ist eine Webanwendung zur Verwaltung und Überwachung von Minecraft-Servern. Die Software richtet sich an Server-Administratoren und bietet eine grafische Benutzeroberfläche zur Steuerung des Server-Status, zur Verwaltung von Spieler-Whitelists sowie zur Bearbeitung von Server-Konfigurationen und NBT-Daten.

Ausgangslage/ Aufgabenstellung

Die Entwicklung des Projekts basiert auf der Anforderung, die Administration von dedizierten Minecraft-Servern über eine zentrale, webbasierte Benutzeroberfläche zugänglich zu machen. Das Problem bestand darin, isolierte Systemprozesse wie das Docker-Container-Management, das Auslesen von Server-Statistiken sowie die Verwaltung von Konfigurationsdateien und Whitelists in einer einheitlichen Applikation zu konsolidieren. Ziel war die Implementierung einer Client-Server-Architektur, welche eine geschützte Admin-Ansicht mit direktem Zugriff auf die Live-Konsole, Server-Eigenschaften und Spielerdaten bereitstellt.

Verwendete Technologien

Das Projekt ist als Full-Stack-Applikation konzipiert und verwendet netzwerkübergreifend TypeScript für die Typsicherheit. Im Frontend kommt React als primäre UI-Bibliothek zum Einsatz, während Vite den Build-Prozess und die lokale Entwicklungsumgebung steuert. Die serverseitige Logik ist in Node.js implementiert und strukturiert sich in dedizierte Routen und Services. Für die Interaktion mit dem Host-System wird programmatisch ein Docker-Service genutzt, um die zugrundeliegenden Server-Container zu verwalten. Spezifische binäre Datenformate des Spiels werden über einen dedizierten NBT-Service verarbeitet, während die asynchrone Kommunikation zwischen Client und Server über eine zentrale API-Schnittstelle abgewickelt wird.

Umsetzung/Herausforderungen

Die Architektur folgt einer strikten Trennung zwischen einer Frontend-Applikation und einem serverseitigen Backend. Im Frontend ist die Präsentationslogik in wiederverwendbare UI-Elemente wie Spielerkarten, Server-Statistiken und Admin-spezifische Komponenten für Konfigurations-Editoren aufgeteilt. Die Navigationsstruktur unterscheidet primär zwischen öffentlichen Home-

Ansichten, einer Login-Maske und einem geschützten Administrationsbereich. Backend-seitig wird eine Service-orientierte Architektur angewandt, welche die Geschäftslogik in spezialisierte Module für Authentifizierung, Docker-Interaktion, Minecraft-spezifische Aufgaben und UUID-Auflösung unterteilt. Die technische Integration der Live-Konsole und die dynamische Manipulation der Whitelist sowie der Server-Eigenschaften erforderten dedizierte Schnittstellen, die durch spezialisierte Routing-Module für Spieler, Server und Authentifizierung abgebildet werden.

Ergebnisse und Erfolge

Das finale System stellt eine funktionstüchtige Verwaltungsplattform dar, die Client- und Server-Komponenten erfolgreich integriert. Kernfunktionen wie die Steuerung des Server-Status, das Auslesen einer textbasierten Live-Konsole sowie die direkte grafische Manipulation der Whitelist und der Server-Eigenschaften wurden in spezifischen Admin-Komponenten realisiert. Das Frontend kommuniziert über definierte API-Endpunkte mit den Backend-Services, um Spielerdaten abzurufen und die Docker-basierten Serverprozesse zu steuern.

Erkenntnisse/Lernerfahrung

Die Verzeichnisstruktur des Repositories demonstriert etablierte Praktiken in der Full-Stack-Entwicklung mit TypeScript, insbesondere die klare Kapselung von Authentifizierungsmechanismen und spielspezifischen Diensten in getrennten Schichten. Der Code zeigt auf, wie Systemzugriffe auf externe Abhängigkeiten wie Docker und proprietäre Dateiformate wie NBT durch dedizierte Service-Klassen abstrahiert werden können, um die Skalierbarkeit und Wartbarkeit der API zu gewährleisten. Darüber hinaus illustriert die Architektur des Frontends eine strikte Trennung von seitenübergreifenden Elementen, ansichtsspezifischen Hauptseiten und modularen Administrationswerkzeugen innerhalb einer modernen React- und Vite-Build-Umgebung.

Referenzen und Links

<https://github.com/kiraa1q/stalkr>

Pérez Loris
IM24A
22.04.2025
40-50 h

